Sakib Dewan Pranto

1000054037

WebAuthn Report

1. How to Run

After cloning the repository, run "npm install" to install dependencies, then "npm run build" to compile the TypeScript code. Manually copy the HTML templates with "xcopy /E /I src\templates dist\templates" on Windows or "cp -r src/templates dist/templates" on Mac/Linux.

Open two terminals. In the first, start the Firebase emulators with "npm run emulator". In the second, start the server with "$env:NODE_ENV="localhost"; node dist/server.mjs" on Windows PowerShell, or "NODE_ENV=localhost node dist/server.mjs" on Mac/Linux.

Open "http://localhost:8080" in your browser. You can register a WebAuthn credential (fingerprint/security key) and authenticate with it. All data stays locally on your machine. To view or tamper with stored credentials, open the Firestore emulator UI at "http://localhost:4000".

---

2. Registration (Attestation) Verification

The server's “/webauthn/registerResponse” handler verifies a new credential by performing these checks. It first validates the challenge in the “clientDataJSON” to prevent replay attacks, confirming it matches the one stored in the session. It then checks the origin in the “clientDataJSON” to ensure the request came from the legitimate application. Finally, it extracts the credential public key and credential ID from the authenticator data within the "attestationObject" and stores them in the session, associated with the user, for use in future authentications.

---

3. Authentication (Assertion) Verification

The server's "/webauthn/authResponse" handler verifies a user's identity by performing a series of checks. It looks up the stored credential public key using the provided "credentialID". It then verifies the

challenge and origin in the "clientDataJSON" to ensure the request is fresh and from the correct source. The core step is the cryptographic signature verification, using the stored public key to validate the signature over the "authenticatorData" and hashed "clientDataJSON", proving possession of the private key. It also checks the User Present (UP) flag in the "authenticatorData" to confirm a human was involved and optionally runs a signature counter check to detect potential authenticator cloning by ensuring the "signCount" is greater than the previously stored value.

---

4. Implementation

Based on the provided "server.txt" and "main.ts" files, here is a report describing the WebAuthn implementation.

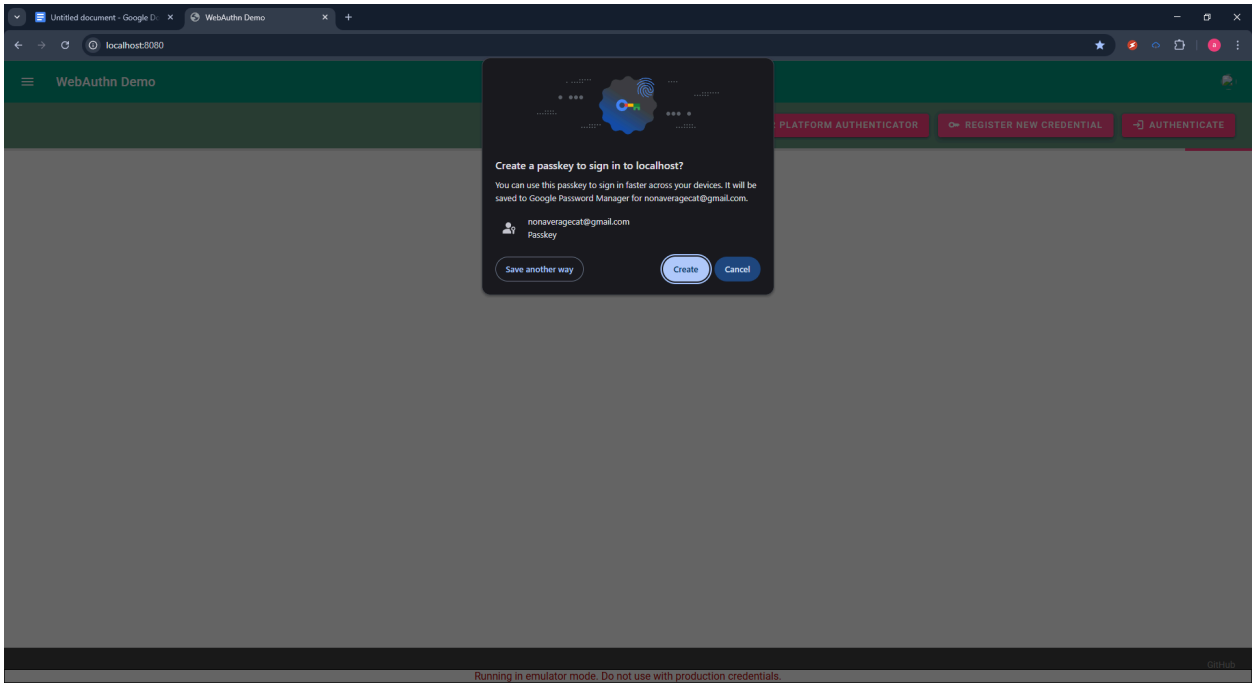
The Relying Party (RP) configuration is managed dynamically through the "config" object imported from "./libs/config.mjs", which supplies details like the server's "origin". This origin is critical and is used in endpoints that serve Digital Asset Links, specifically "/.well-known/assetlinks.json", to declare the relationship between the web application and any corresponding Android apps for credential sharing. The server's core configuration, including session management via "initializeSession()", is set up in "server.txt", while the client-side initialization in "main.ts" connects to local Firebase emulators for authentication, decoupling the identity provider from the WebAuthn process itself.

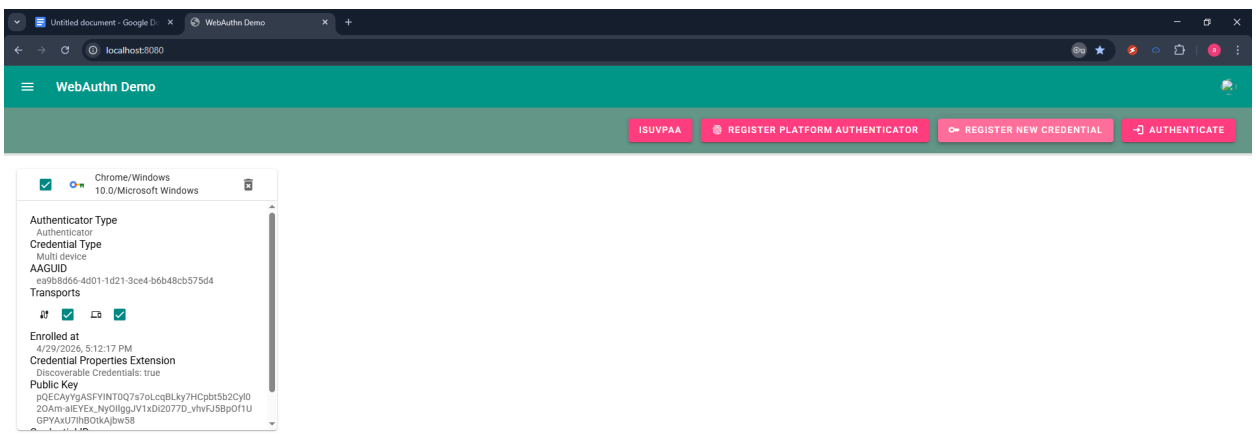
The registration flow is a two-step ceremony initiated by the client. First, the "registerCredential" function in "main.ts" sends user options to the "/webauthn/registerRequest" endpoint to fetch a "PublicKeyCredentialCreationOptionsJSON" object from the server, which contains the critical, unique challenge. The client decodes the base64url-encoded challenge and any existing credentials to be excluded, then calls "navigator.credentials.create()". The resulting attestation, including the "attestationObject" and "clientDataJSON", is base64url-encoded and sent back to the "/webauthn/registerResponse" endpoint for verification. The authentication flow mirrors this. The "authenticate" function fetches a challenge from "/webauthn/authRequest", invokes "navigator.credentials.get()", and sends the signed assertion, containing the "authenticatorData", "clientDataJSON", and "signature", to "/webauthn/authResponse" for server-side validation.

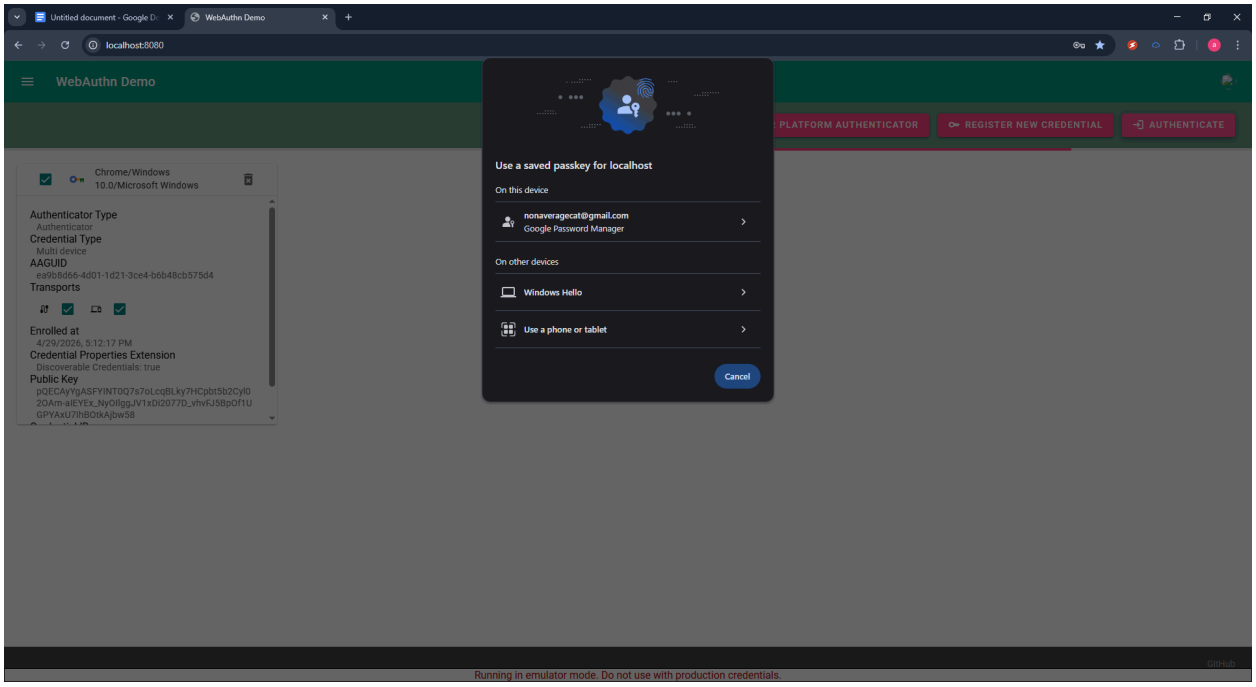
On the server, the cryptographic verification is handled within the "webauthn.mjs" library's request handlers. During registration, the server must decode the "clientDataJSON" and verify that its embedded challenge matches the one originally issued and stored in the user's session, and that the origin is correct, preventing replay and cross-site forgery attacks. The authenticator's attestation statement format is checked and, if not ""none"", may be cryptographically verified. Crucially, the public key is then extracted from the "attestationObject"'s authenticator data and stored. For authentication, after looking up the stored public key via the "credentialID", the server again verifies the challenge and origin. The central verification is the cryptographic signature check: using the stored public key to validate the assertion's "signature" over the concatenation of the "authenticatorData" and the SHA-256 hash of the "clientDataJSON". Finally, the server checks the authenticator data's flags for User Presence and can optionally validate the signature counter to detect cloned authenticators.

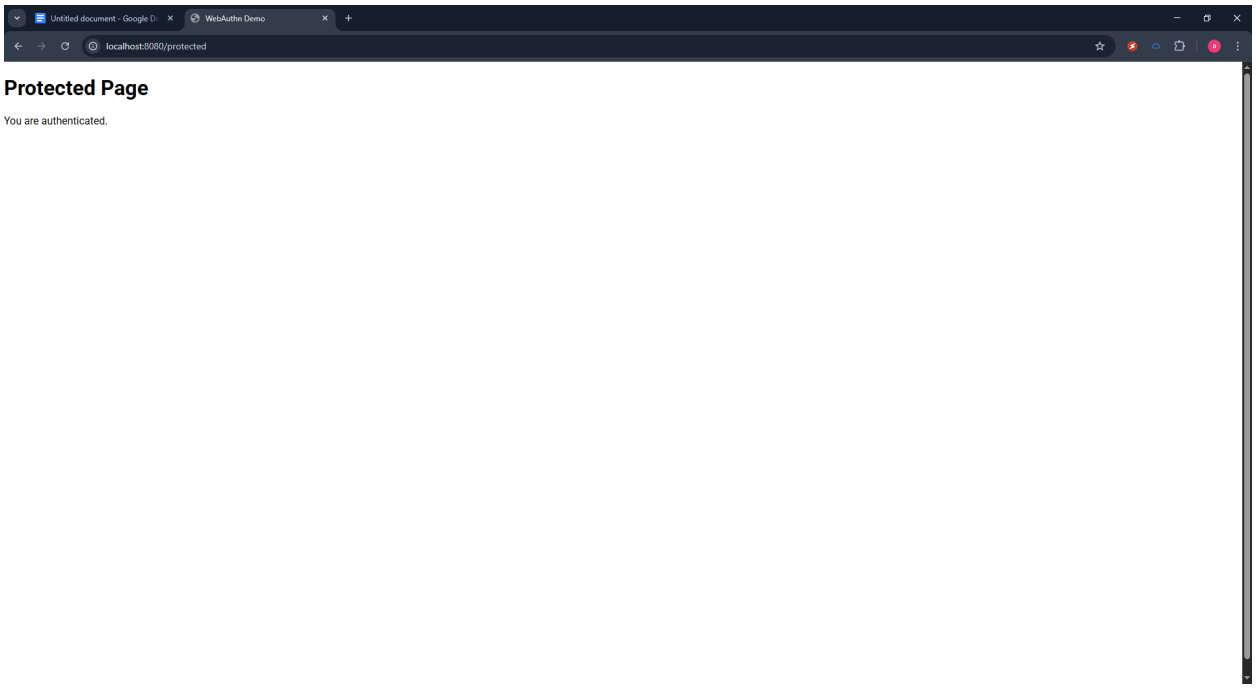
5. Screen shots

Chrome screen shots :

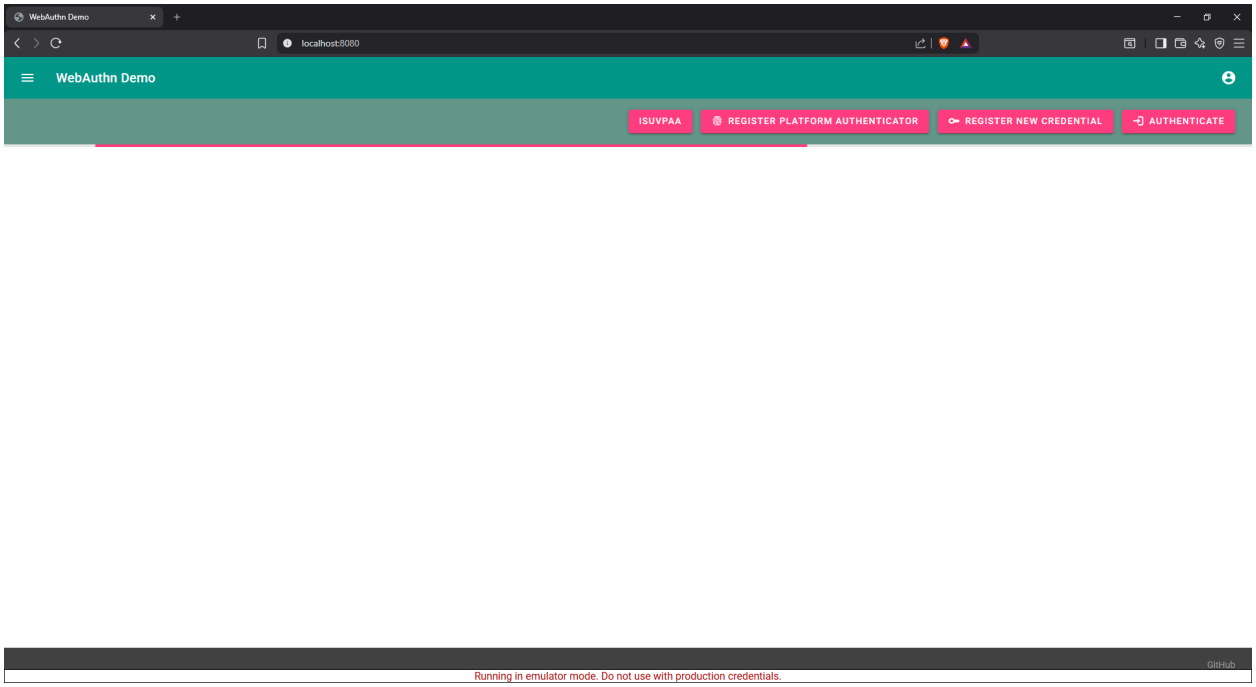


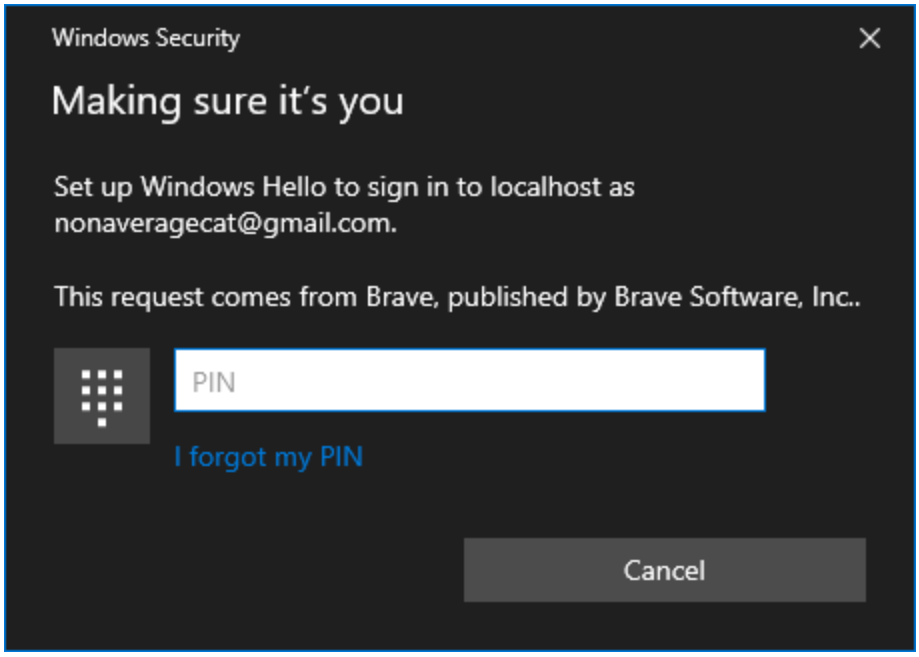


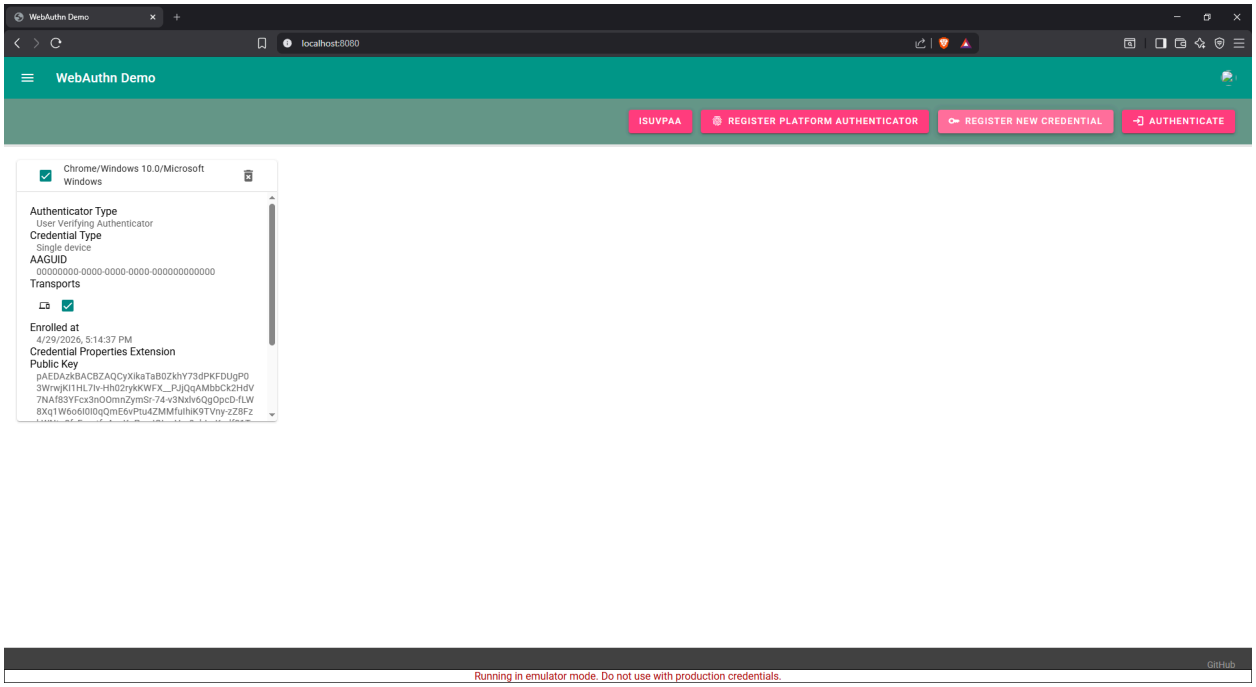

Untitled document - Google D
WebAuthn Demo
localhost:8080
WebAuthn Demo
PLATFORM AUTHENTICATOR
REGISTER NEW CREDENTIAL
AUTHENTICATE
Chrome/Windows
10.0/Microsoft Windows
Authenticator Type
Authenticator
Credential Type
Multi device
AAGUID
ea9b8d66-4d01-1d21-3ce4-b6b48cb575d4
Transports
Enrolled at
4/29/2026, 5:12:17 PM
Credential Properties Extension
Discoverable Credentials: true
Public Key
pQECAyYgASFYINT0Q7s7oLcqBLky7HCpbt5b2Cyl0
2OAm-alEYEx_NyOIlggJV1xDi2077D_vhvFJ5BpOf1U
GPYAxU7lhBOtkAJbw58
Use a saved passkey for localhost
On this device
nonaveragecat@gmail.com
Google Password Manager
On other devices
Windows Hello
Use a phone or tablet
Cancel
Running in emulator mode. Do not use with production credentials.

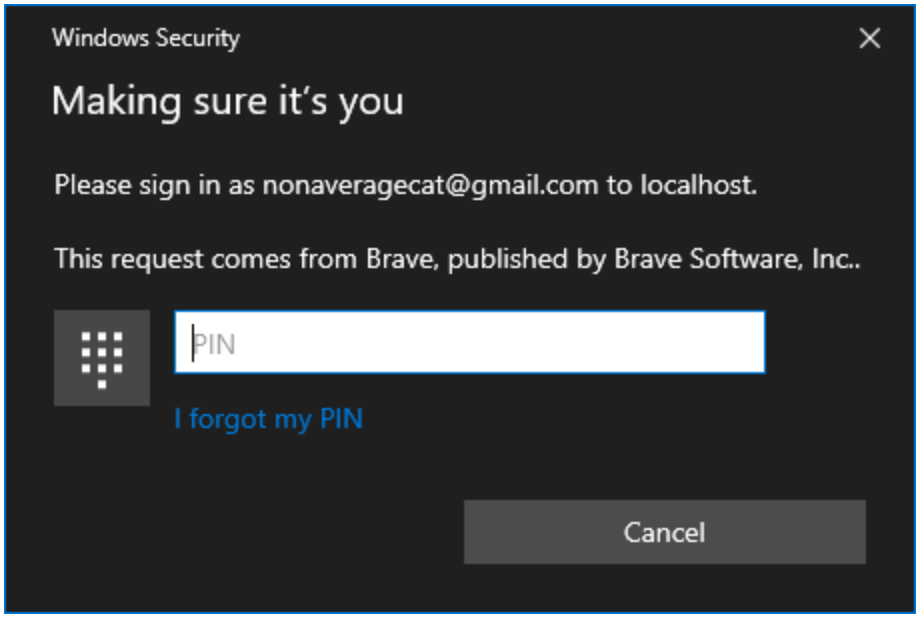

Untitled document - Google D
WebAuthn Demo
localhost:8080/protected
Protected Page
You are authenticated.

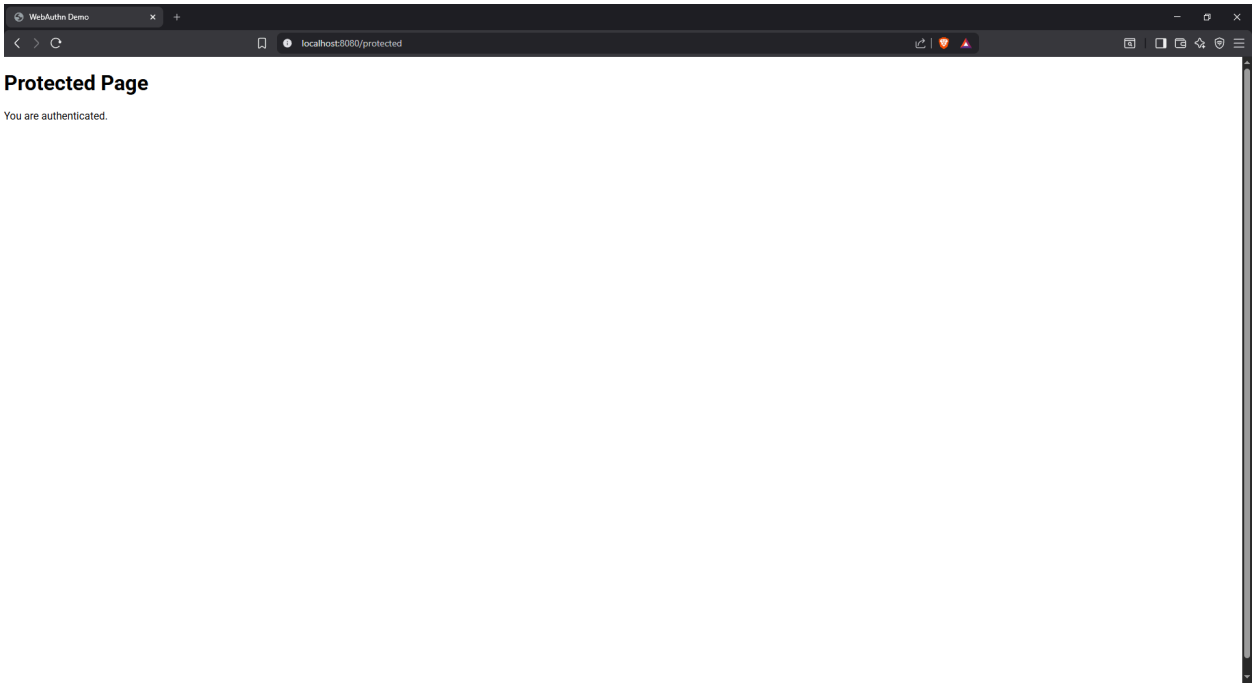
Brave screen shots



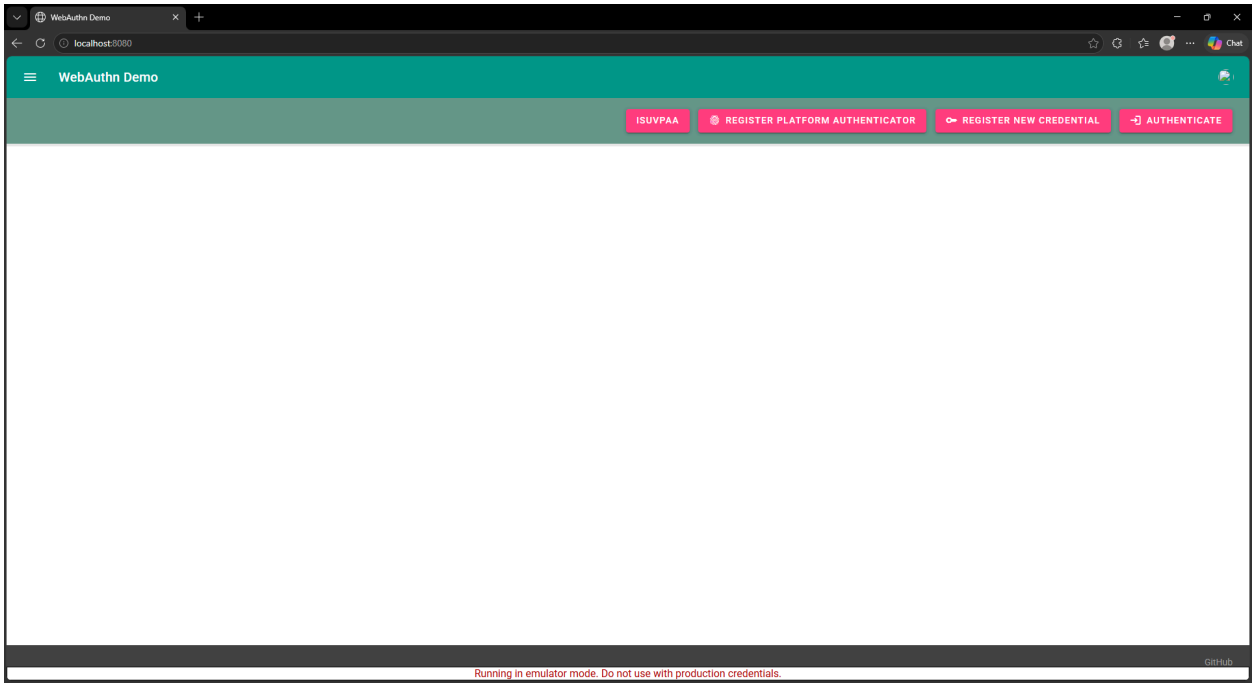

Windows Security
Making sure it's you
Set up Windows Hello to sign in to localhost as nonaveragecat@gmail.com.
This request comes from Brave, published by Brave Software, Inc..
PIN
I forgot my PIN
Cancel

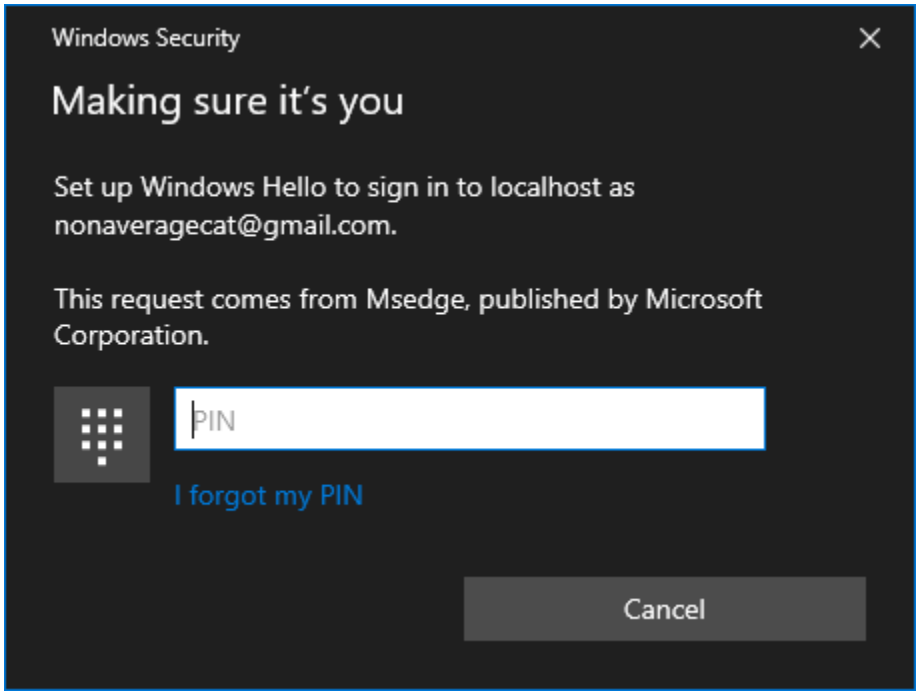

WebAuthn Demo
localhost:8080
WebAuthn Demo
ISUVPAA
REGISTER PLATFORM AUTHENTICATOR
REGISTER NEW CREDENTIAL
AUTHENTICATE
Chrome/Windows 10.0/Microsoft Windows
Authenticator Type
User Verifying Authenticator
Credential Type
Single device
AAGUID
00000000-0000-0000-0000-000000000000
Transports
Enrolled at
4/29/2026, 5:14:37 PM
Credential Properties Extension
Public Key
pAEDAzkBACBZAQCyXikaTaB0ZkhY73dPKFDUgP0
3WrwjK1HL7Iv-Hh02rykKWFX_PJjQqAMbbCk2HdV
7NAf83YFcx3nOOmnZymSr-74-v3NxIv6QgOpcD-fLW
8Xq1W6o6I0I0qQmE6vPtu4ZMMfulhIK9TVny-zZ8Fz
GitHub
Running in emulator mode. Do not use with production credentials.

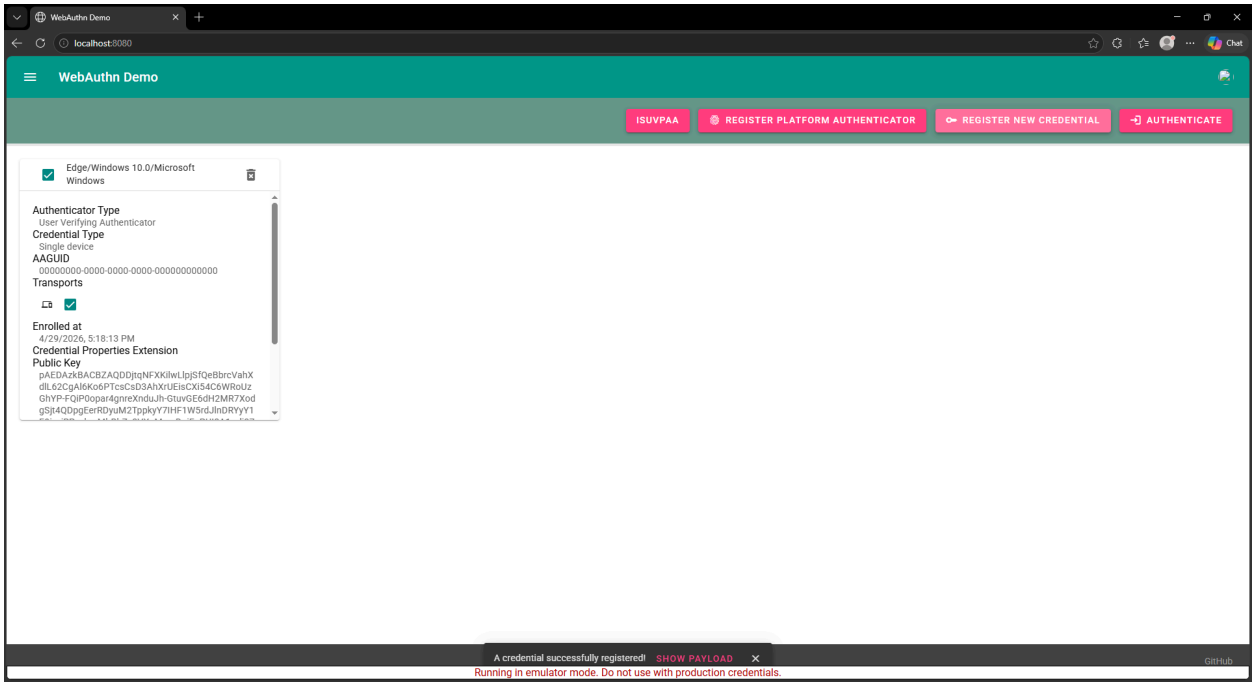

Windows Security
Making sure it's you
Please sign in as nonaveragecat@gmail.com to localhost.
This request comes from Brave, published by Brave Software, Inc..
PIN
I forgot my PIN
Cancel

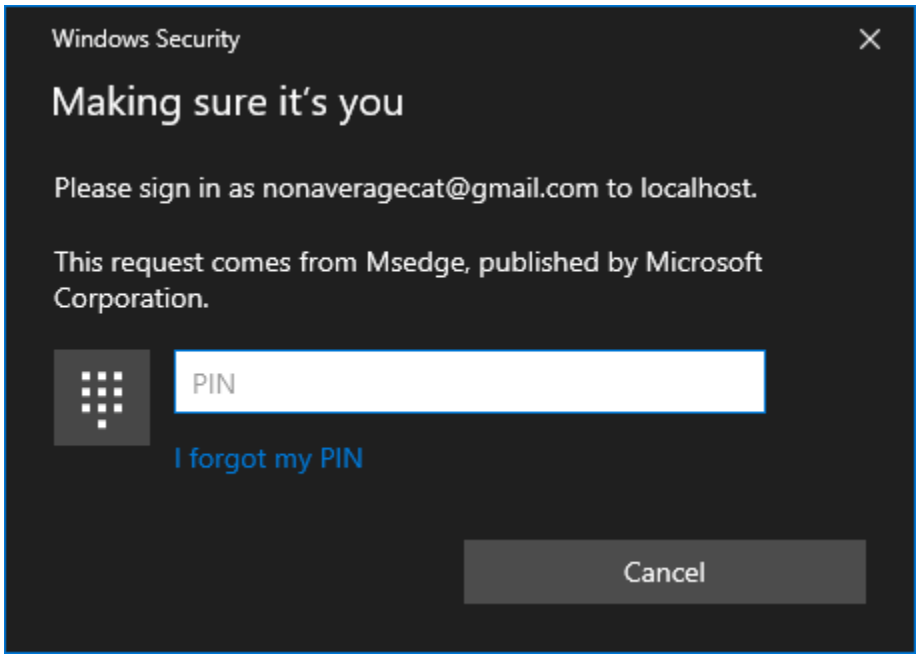

WebAuthn Demo
localhost:8080/protected
Protected Page
You are authenticated.

Edge screen shots



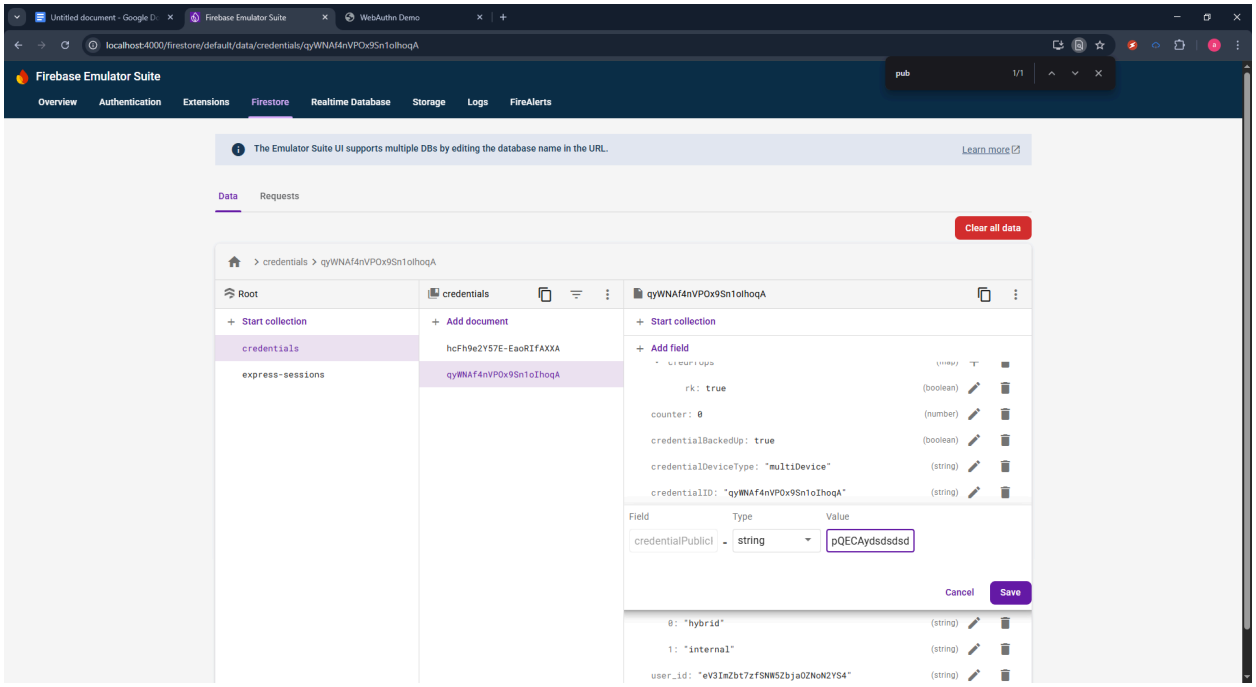


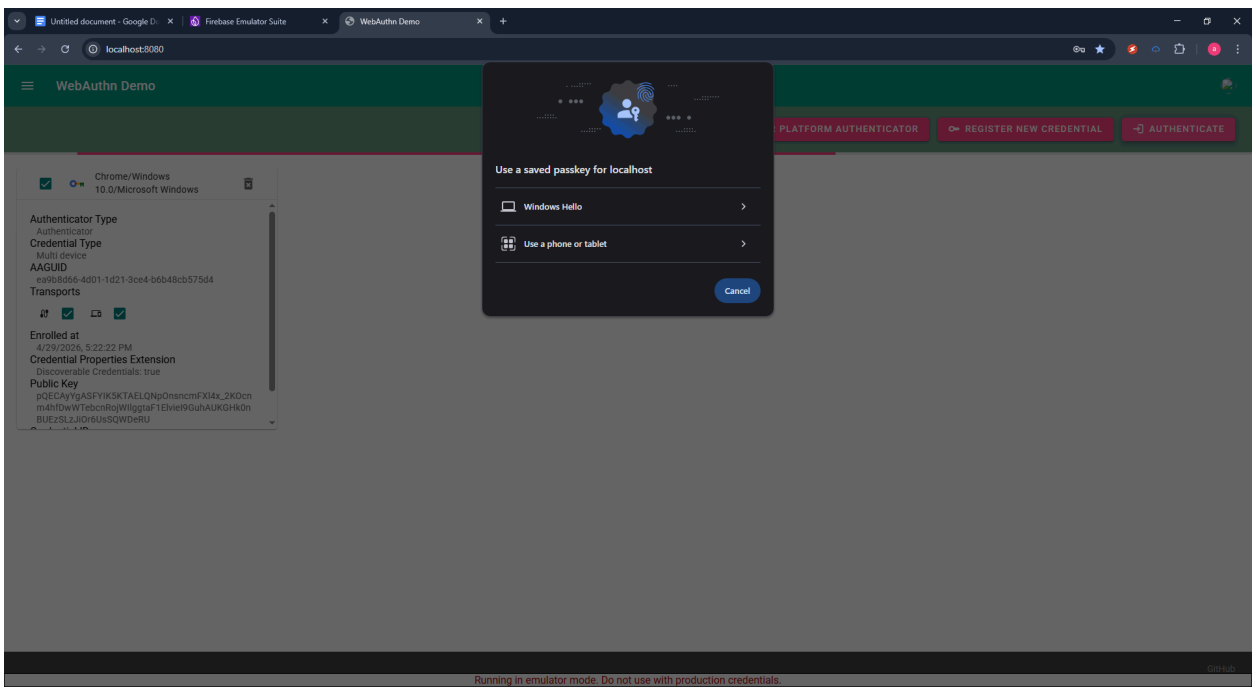

WebAuthn Demo
localhost:8080
WebAuthn Demo
ISUVPAA
REGISTER PLATFORM AUTHENTICATOR
REGISTER NEW CREDENTIAL
AUTHENTICATE
Edge/Windows 10.0/Microsoft Windows
Authenticator Type
User Verifying Authenticator
Credential Type
Single device
AAGUID
00000000-0000-0000-0000-000000000000
Transports
Enrolled at
4/29/2026, 5:18:13 PM
Credential Properties Extension
Public Key
pAEDAzkBACBZAQDDjtqNFXKilwLlpjSfQeBbrcVahX
dlL62CgAI6Ko6PTcsCsD3AhXrUEisCXi54C6WRoUz
GhYP-FQlP0opar4gnreXnduJh-GtuvGE6dH2MR7Xod
gSjt4QDpgEerRDyuM2TppkyY7IHF1W5rdJlnDRYyY1
A credential successfully registered!
SHOW PAYLOAD
Running in emulator mode. Do not use with production credentials.
GitHub


Windows Security
Making sure it's you
Please sign in as nonaveragecat@gmail.com to localhost.
This request comes from Msedge, published by Microsoft Corporation.
PIN
I forgot my PIN
Cancel

WebAuthn Demo
localhost:8080/protected
Chat
Protected Page
You are authenticated.

Negative test attempt chrome:





Github Link : https://github.com/Glitch1258/webAuthDemo.git